Name: Jayesh Deshmukh
Class: D15C
Batch: C
Roll No: 57

MLDL – Experiment 7

Aim

Build an Artificial Neural Network (ANN) using Keras/TensorFlow for binary classification and evaluate model performance with and without hyperparameter tuning.

---

1. Dataset Source

- Dataset: Pima Indians Diabetes Dataset
- Repository: UCI Machine Learning Repository
- Kaggle: https://www.kaggle.com/datasets/uciml/pima-indians-diabetes-database
- File used: diabetes.csv

---

2. Dataset Description

The Pima Indians Diabetes dataset originates from the National Institute of Diabetes and Digestive and Kidney Diseases. It is used to predict the onset of diabetes in female patients of Pima Indian heritage based on diagnostic measurements. It is a standard benchmark for binary medical classification.

Key properties:

- Dataset size: 768 samples × 9 columns (8 features + 1 target)
- Target variable: Outcome (1 = diabetes present, 0 = absent)
- Class distribution: 500 negative (no diabetes), 268 positive (diabetes) — moderately imbalanced
- Missing values: None (zero-values in certain columns represent medically implausible zeros but are retained as-is)

Feature columns:

- Pregnancies: Number of times pregnant
- Glucose: Plasma glucose concentration (2-hour oral glucose tolerance test)
- BloodPressure: Diastolic blood pressure (mm Hg)
- SkinThickness: Triceps skinfold thickness (mm)

- Insulin: 2-hour serum insulin (μU/ml)
- BMI: Body mass index (weight kg / height m^2)
- DiabetesPedigreeFunction: Genetic influence score based on family history
- Age: Age in years

---

3. Mathematical Formulation of the Algorithm

An Artificial Neural Network (ANN) is a computational model inspired by biological neural networks. It is composed of an input layer, one or more hidden layers, and an output layer. Each layer applies a linear transformation followed by a non-linear activation function.

3.1 Neuron Computation (Single Layer)

For layer l with weight matrix W^l and bias b^l:

$$z^l = W^l \cdot a^{l-1} + b^l$$

$$a^l = f^l(z^l)$$

where a^l is the activation output and f^l is the activation function at layer l.

3.2 Activation Functions

ReLU (hidden layers) — introduces non-linearity while avoiding the vanishing gradient problem:

$$ReLU(z) = max(0, z)$$

Sigmoid (output layer) — squashes output to [0,1] for binary probability estimation:

$$\sigma(z) = 1 / (1 + e^{-r})$$

3.3 Loss Function (Binary Cross-Entropy)

For m training samples with true labels y and predictions ŷ:

$$J = -(1/m) \Sigma_i [\, y_i \log(\hat{y}_i) + (1-y_i) \log(1-\hat{y}_i) \,]$$

3.4 Backpropagation

Gradients of the loss with respect to each layer's weights and biases are computed by the chain rule, propagating error backwards from output to input:

$$\partial J/\partial W^l = (1/m) \cdot \partial J/\partial z^l \cdot (a^{l-1})^T$$

$$\partial J/\partial b^l = (1/m) \cdot \Sigma_i \, \partial J/\partial z^l$$

3.5 Adam Optimizer

Adam (Adaptive Moment Estimation) maintains per-parameter learning rates using estimates of first and second moments of gradients:

$$m_t = \beta_1 \; m_{t-1} + (1-\beta_1)\, g_t$$

$$v_t = \beta_2 \; v_{t-1} + (1-\beta_2)\, g_t^2$$

$$\theta_t = \theta_{t-1} - \alpha \cdot \hat{m}_t / (\sqrt{\hat{v}_t} + \varepsilon)$$

where $\hat{m}_t$ and $\hat{v}_t$ are bias-corrected estimates, α is the learning rate, and ε prevents division by zero. Default: $\beta_1 = 0.9$, $\beta_2 = 0.999$.

3.6 Dropout Regularization

During training, each neuron is independently zeroed out with probability p (the dropout rate). This prevents co-adaptation of neurons and reduces overfitting. At inference time, all neurons are active and weights are scaled accordingly.

---

4. Algorithm Limitations

1. Data Hungry

ANNs typically require large amounts of labelled training data to learn meaningful representations. On small datasets like Pima Indians (768 samples), the network may underfit or show high variance between runs, as there are insufficient examples to reliably estimate complex non-linear boundaries.

2. Hyperparameter Sensitivity

Performance is heavily influenced by architecture choices (number of layers, neurons per layer), learning rate, batch size, dropout rate, and number of epochs. Poor choices can lead to slow convergence, vanishing gradients, or overfitting. Systematic tuning is essential.

3. Prone to Overfitting

A network with too many parameters relative to training samples will memorise the training data. Regularisation techniques such as dropout, L2 weight decay, and early stopping are necessary to ensure generalisation.

4. Black-Box Nature

Unlike decision trees or logistic regression, ANNs do not produce human-interpretable rules or direct feature importance scores. Explainability techniques (SHAP, LIME) are required to understand individual predictions.

5. Computationally Expensive

Training requires repeated forward and backward passes through all layers over many epochs. While small networks on tabular data are manageable, larger architectures and datasets demand GPU/TPU acceleration.

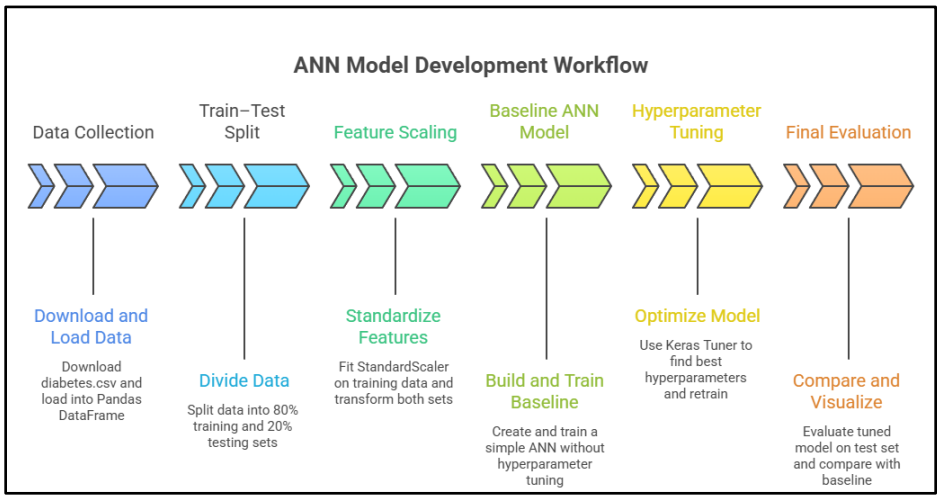
6. Sensitivity to Feature Scaling

Gradient-based optimisation converges poorly when input features are on very different scales. StandardScaler normalisation is non-negotiable before training an ANN.

---

5. Methodology / Workflow

1. **Data Collection**
   - Download diabetes.csv from the Kaggle Pima Indians Diabetes dataset.
   - Load into a Pandas DataFrame; confirm shape (768 × 9) and class distribution.
2. **Train–Test Split**
   - Split into 80% training (614 samples) and 20% testing (154 samples) with stratify=y and random_state=42.
3. **Feature Scaling**
   - Fit StandardScaler on the training set; transform both training and test sets to zero mean and unit variance.
4. **Baseline ANN Model (Without Tuning)**
   - Build a sequential ANN: Input(8) → Dense(64, ReLU) → Dense(32, ReLU) → Dense(1, Sigmoid).
   - Compile with Adam(lr=0.001) and binary cross-entropy loss. Train for 100 epochs, batch size 32, 10% validation split.
   - Evaluate: accuracy, classification report (per-class precision, recall, F1), confusion matrix, training curves.
5. **Hyperparameter Tuning (Keras Tuner)**
   - Define a model builder function parameterised over: number of hidden layers (1–3), units per layer (16/32/64/128), dropout rate per layer (0.0–0.4), and learning rate (1e–4 to 5e–3).
   - Run RandomSearch with max_trials=20, objective='val_accuracy', EarlyStopping(patience=10).
   - Extract best hyperparameters; retrain the best model on full training data.
6. **Final Evaluation**
   - Evaluate the tuned model on the same test set. Compare accuracy, per-class metrics, and confusion matrix against the baseline.
   - Visualise training accuracy and loss curves for both models.



6. Performance Analysis

Model: Sequential ANN — Input(8) → Dense(64, ReLU) → Dense(32, ReLU) → Dense(1, Sigmoid)

Total Parameters: 2,689 | Optimizer: Adam(lr=0.001) | Epochs: 100 | Batch Size: 32

Train/Test Split: 80% / 20% | Test Set: 154 samples (100 negative, 54 positive)

Metric	No Diabetes	Diabetes	Overall
Precision	0.78	0.57	—
Recall	0.75	0.61	—
F1-Score	0.77	0.59	—
Support	100	54	154
Accuracy	—	—	70.13%

1. **No Diabetes (Class 0)**

   Precision of 0.78 and recall of 0.75 reflect moderate performance on the majority class. Of 100 truly healthy patients, 75 are correctly identified (TN=75) and 25 are incorrectly predicted as diabetic (FP=25).

2. **Diabetes (Class 1)**

   Precision of 0.57 means that when the model predicts diabetes, it is correct only 57% of the time. Recall of 0.61 means 39% of true diabetic patients are missed (FN=21, TP=33). The low F1-score of 0.59 reflects the difficulty of learning the minority class boundary with the default architecture.

3. **Overall**

   The baseline ANN achieves 70.13% accuracy, correctly classifying 108 out of 154 test samples. The class imbalance (500 vs 268) is a contributing factor — the model learns a stronger representation of the majority class. The untuned architecture (64→32 units) and default learning rate provide a reasonable but improvable starting point.

7. Hyperparameter Tuning

Before Tuning — Baseline Architecture

Parameter	Baseline Value
num_layers	2 (fixed)
units (layer 1)	64

units (layer 2)	32
dropout	None
learning_rate	0.001 (Adam default)
epochs	100
batch_size	32

Baseline Test Accuracy: 70.13% | Diabetes F1: 0.59 | Diabetes Recall: 0.61

Keras Tuner — RandomSearch Configuration:

- num_layers: Int in [1, 3]
- units per layer: Choice of [16, 32, 64, 128]
- dropout per layer: Float in [0.0, 0.4] (step 0.1)
- learning_rate: Choice of [1e–4, 5e–4, 1e–3, 5e–3]
- max_trials = 20, objective = 'val_accuracy', seed = 42
- EarlyStopping: monitor='val_loss', patience=10, restore_best_weights=True

Best Hyperparameters Found:

Parameter	Best Value
num_layers	2
units (layer 1)	128
dropout (layer 1)	0.2
units (layer 2)	32
dropout (layer 2)	0.0
learning_rate	0.005
Epochs (early stopped at)	13

After Tuning — Results:

Metric	No Diabetes	Diabetes	Overall
Precision	0.79	0.65	—
Recall	0.83	0.59	—
F1-Score	0.81	0.62	—
Support	100	54	154
Accuracy	—	—	74.68%

Key Observations:

- Accuracy improved from 70.13% → 74.68% (+4.55 percentage points)
- Wider first layer (128 units vs 64) allows the network to capture richer feature interactions from the 8 input features, particularly combinations involving Glucose, BMI, and Age
- Dropout of 0.2 in the first layer acts as a regulariser — randomly disabling 20% of neurons during training prevents the larger layer from overfitting on the small dataset
- Higher learning rate (0.005 vs 0.001) with early stopping led to fast convergence in just 13 epochs, avoiding the over-training seen in the 100-epoch baseline
- No Diabetes recall improved from 0.75 → 0.83 — false positives reduced from 25 → 17, meaning fewer healthy patients are incorrectly flagged
- Diabetes precision improved significantly from 0.57 → 0.65 — when the tuned model predicts diabetes, it is correct 65% of the time vs 57% before
- Diabetes recall decreased marginally from 0.61 → 0.59 (FN: 21 → 22) — a minor trade-off for the overall accuracy and precision gains

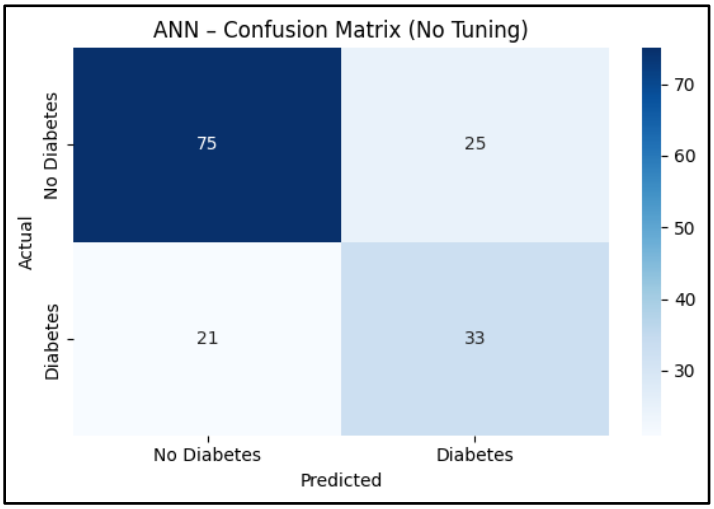
---

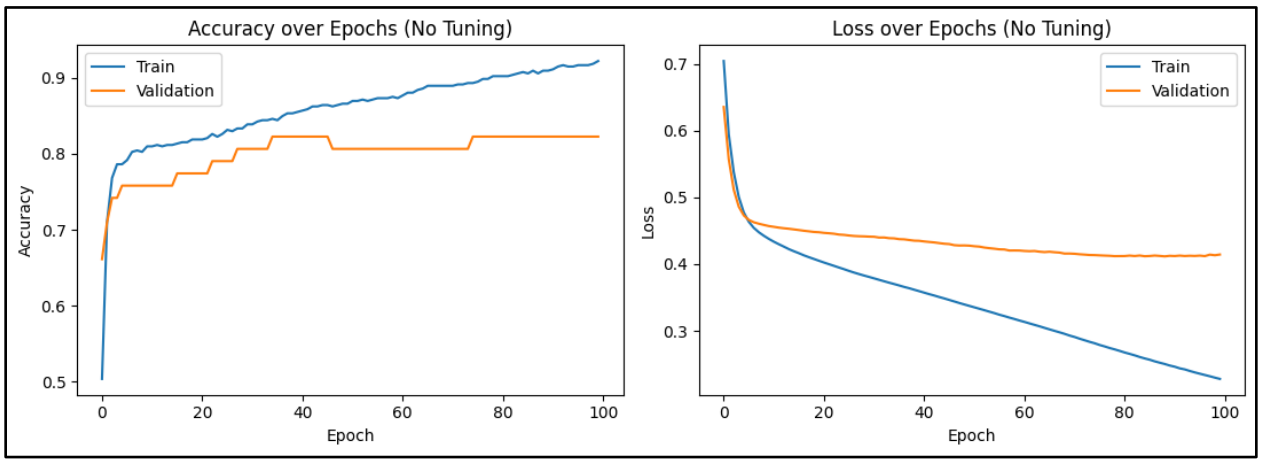
8. Code and Output – Without Tuning

```
# Artificial Neural Network – Without Hyperparameter Tuning
# Dataset: Pima Indians Diabetes (diabetes.csv)

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers

tf.random.set_seed(42)
np.random.seed(42)

# Load dataset
df = pd.read_csv("diabetes.csv")
print("Shape:", df.shape)
print(df.head())
print("\nClass distribution (Outcome):")
print(df["Outcome"].value_counts())

# Features and target
X = df.drop("Outcome", axis=1)
y = df["Outcome"]

# Train-test split (80/20, stratified)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
print("\nTrain shape:", X_train.shape, "| Test shape:", X_test.shape)

# Feature scaling
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled  = scaler.transform(X_test)
```

```
# Baseline ANN: Input(8) → Dense(64,relu) → Dense(32,relu) → Dense(1,sigmoid)
model = keras.Sequential([
    layers.Input(shape=(X_train_scaled.shape[1],)),
    layers.Dense(64, activation='relu'),
    layers.Dense(32, activation='relu'),
    layers.Dense(1,  activation='sigmoid')
])

model.compile(
    optimizer=keras.optimizers.Adam(learning_rate=0.001),
    loss='binary_crossentropy',
    metrics=['accuracy']
)

model.summary()

# Train
history = model.fit(
    X_train_scaled, y_train,
    epochs=100,
    batch_size=32,
    validation_split=0.1,
    verbose=1
)

# Evaluate
y_pred_prob = model.predict(X_test_scaled).flatten()
y_pred = (y_pred_prob >= 0.5).astype(int)

acc = accuracy_score(y_test, y_pred)
print("\n========== ANN (No Tuning) ==========")
print(f"Accuracy: {acc * 100:.2f}%")
print("\nClassification Report:")
print(classification_report(y_test, y_pred,
      target_names=["No Diabetes", "Diabetes"]))
cm = confusion_matrix(y_test, y_pred)
print("Confusion Matrix:\n", cm)

# Confusion matrix plot
plt.figure(figsize=(6, 4))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=["No Diabetes", "Diabetes"],
            yticklabels=["No Diabetes", "Diabetes"])
plt.title("ANN – Confusion Matrix (No Tuning)")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.tight_layout()
plt.show()

# Training curves
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(11, 4))
ax1.plot(history.history['accuracy'],     label='Train')
ax1.plot(history.history['val_accuracy'], label='Validation')
ax1.set_title('Accuracy over Epochs (No Tuning)')
ax1.set_xlabel('Epoch')
ax1.set_ylabel('Accuracy')
ax1.legend()
ax2.plot(history.history['loss'],     label='Train')
ax2.plot(history.history['val_loss'], label='Validation')
ax2.set_title('Loss over Epochs (No Tuning)')
ax2.set_xlabel('Epoch')
ax2.set_ylabel('Loss')
ax2.legend()
plt.tight_layout()
plt.show()

print("\nArchitecture: Input(8) → Dense(64,relu) → Dense(32,relu) → Dense(1,sigmoid)")
print("Optimizer: Adam(lr=0.001) | Epochs: 100 | Batch Size: 32")
```

```
Epoch 90/100
18/18 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.9094 - loss: 0.2480 - val_accuracy: 0.8226 - val_loss: 0.4124
Epoch 91/100
18/18 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.9112 - loss: 0.2462 - val_accuracy: 0.8226 - val_loss: 0.4121
Epoch 92/100
18/18 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.9149 - loss: 0.2438 - val_accuracy: 0.8226 - val_loss: 0.4128
Epoch 93/100
18/18 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.9167 - loss: 0.2422 - val_accuracy: 0.8226 - val_loss: 0.4122
Epoch 94/100
18/18 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.9149 - loss: 0.2398 - val_accuracy: 0.8226 - val_loss: 0.4125
Epoch 95/100
18/18 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.9149 - loss: 0.2377 - val_accuracy: 0.8226 - val_loss: 0.4123
Epoch 96/100
18/18 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.9167 - loss: 0.2357 - val_accuracy: 0.8226 - val_loss: 0.4127
Epoch 97/100
18/18 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.9167 - loss: 0.2338 - val_accuracy: 0.8226 - val_loss: 0.4120
Epoch 98/100
18/18 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.9167 - loss: 0.2319 - val_accuracy: 0.8226 - val_loss: 0.4142
Epoch 99/100
18/18 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.9185 - loss: 0.2299 - val_accuracy: 0.8226 - val_loss: 0.4133
Epoch 100/100
18/18 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.9221 - loss: 0.2280 - val_accuracy: 0.8226 - val_loss: 0.4144
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 13ms/step
```





9. Code and Output – With Tuning

Artificial Neural Network – With Hyperparameter Tuning (Keras Tuner)

Dataset: Pima Indians Diabetes (diabetes.csv)

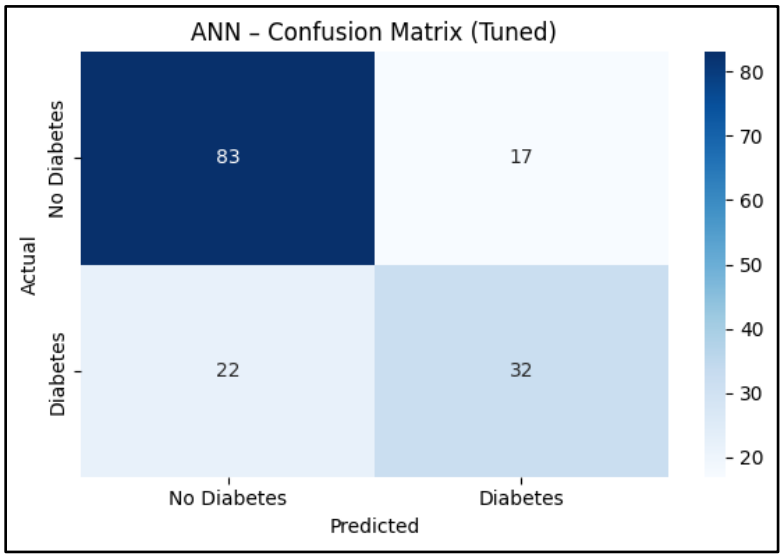
```
!pip install keras-tuner
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import
train_test_split
from sklearn.preprocessing import
StandardScaler
from sklearn.metrics import
accuracy_score, classification_report,
confusion_matrix
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
import keras_tuner as kt

tf.random.set_seed(42)
np.random.seed(42)

# Load and preprocess (identical to
baseline)
df = pd.read_csv("diabetes.csv")
X = df.drop("Outcome", axis=1)
y = df["Outcome"]

X_train, X_test, y_train, y_test =
train_test_split(
    X, y, test_size=0.2, random_state=42,
stratify=y
)

scaler = StandardScaler()
X_train_scaled =
scaler.fit_transform(X_train)
X_test_scaled  =
scaler.transform(X_test)

n_features = X_train_scaled.shape[1]

# Model builder for Keras Tuner
def build_model(hp):
    model = keras.Sequential()
    model.add(layers.Input(shape=(n_fea
tures,)))
    for i in range(hp.Int('num_layers', 1,
3)):
        units = hp.Choice(f'units_{i}', [16,
32, 64, 128])
        model.add(layers.Dense(units,
activation='relu'))
        drop = hp.Float(f'dropout_{i}', 0.0,
0.4, step=0.1)
        if drop > 0.0:
            model.add(layers.Dropout(drop))

    model.add(layers.Dense(1,
activation='sigmoid'))

    lr = hp.Choice('learning_rate', [1e-4,
5e-4, 1e-3, 5e-3])
    model.compile(
        optimizer=keras.optimizers.Adam(l
earning_rate=lr),
        loss='binary_crossentropy',
        metrics=['accuracy']
    )
    return model

# Keras Tuner — Random Search (20
trials)
tuner = kt.RandomSearch(
    build_model,
    objective='val_accuracy',
    max_trials=20,
    seed=42,
    directory='kt_diabetes',
    project_name='ann_tuning',
    overwrite=True
)

early_stop =
keras.callbacks.EarlyStopping(
    monitor='val_loss', patience=10,
restore_best_weights=True
)

print("Starting hyperparameter search
(20 trials)...")
tuner.search(
    X_train_scaled, y_train,
```

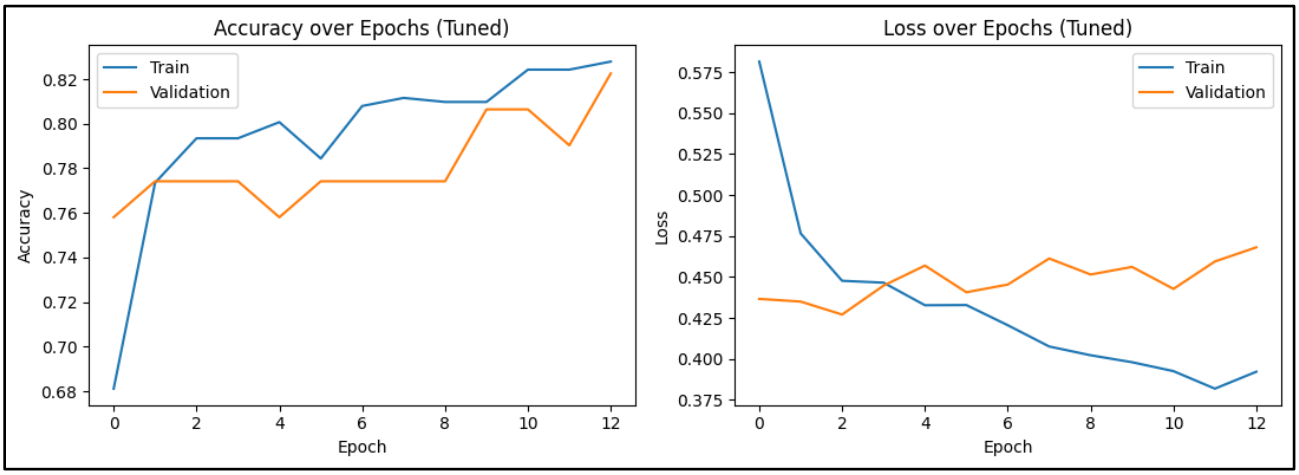
```
    epochs=100,
    batch_size=32,
    validation_split=0.1,
    callbacks=[early_stop],
    verbose=0
)
print("Search complete.")

# Best hyperparameters
best_hps =
tuner.get_best_hyperparameters(num_tr
ials=1)[0]
print("\n========== Best
Hyperparameters Found ==========")
n_layers = best_hps.get('num_layers')
print(f"num_layers    : {n_layers}")
for i in range(n_layers):
    print(f"units_{i}      :
{best_hps.get(f'units_{i}')}")
    print(f"dropout_{i}    :
{best_hps.get(f'dropout_{i}')}")
print(f"learning_rate :
{best_hps.get('learning_rate')}")

# Train best model fully
best_model =
tuner.hypermodel.build(best_hps)
history = best_model.fit(
    X_train_scaled, y_train,
    epochs=100,
    batch_size=32,
    validation_split=0.1,
    callbacks=[early_stop],
    verbose=1
)
print(f"\nTraining stopped at epoch:
{len(history.history['accuracy'])}")

# Evaluate
y_pred_prob =
best_model.predict(X_test_scaled).flatte
n()
y_pred = (y_pred_prob >=
0.5).astype(int)
acc = accuracy_score(y_test, y_pred)
print("\n========== ANN (With Tuning)
==========")
print(f"Accuracy: {acc * 100:.2f}%")
print("\nClassification Report:")
print(classification_report(y_test,
y_pred,
      target_names=["No Diabetes",
"Diabetes"]))
cm = confusion_matrix(y_test, y_pred)
print("Confusion Matrix:\n", cm)

# Confusion matrix plot
plt.figure(figsize=(6, 4))
sns.heatmap(cm, annot=True, fmt='d',
cmap='Blues',
            xticklabels=["No Diabetes",
"Diabetes"],
            yticklabels=["No Diabetes",
"Diabetes"])
plt.title("ANN – Confusion Matrix
(Tuned)")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.tight_layout()
plt.show()

# Training curves
fig, (ax1, ax2) = plt.subplots(1, 2,
figsize=(11, 4))
ax1.plot(history.history['accuracy'],     lab
el='Train')
ax1.plot(history.history['val_accuracy'],
label='Validation')
ax1.set_title('Accuracy over Epochs
(Tuned)')
ax1.set_xlabel('Epoch')
ax1.set_ylabel('Accuracy')
ax1.legend()
ax2.plot(history.history['loss'],     label='T
rain')
ax2.plot(history.history['val_loss'],
label='Validation')
ax2.set_title('Loss over Epochs
(Tuned)')
ax2.set_xlabel('Epoch')
ax2.set_ylabel('Loss')
```

ax2.legend()
plt.tight_layout()

plt.show()

```
Epoch 6/100
18/18 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.7844 - loss: 0.4329 - val_accuracy: 0.7742 - val_loss: 0.4407
Epoch 7/100
18/18 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.8080 - loss: 0.4206 - val_accuracy: 0.7742 - val_loss: 0.4454
Epoch 8/100
18/18 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.8116 - loss: 0.4076 - val_accuracy: 0.7742 - val_loss: 0.4613
Epoch 9/100
18/18 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.8098 - loss: 0.4023 - val_accuracy: 0.7742 - val_loss: 0.4515
Epoch 10/100
18/18 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.8098 - loss: 0.3980 - val_accuracy: 0.8065 - val_loss: 0.4562
Epoch 11/100
18/18 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.8243 - loss: 0.3926 - val_accuracy: 0.8065 - val_loss: 0.4428
Epoch 12/100
18/18 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.8243 - loss: 0.3819 - val_accuracy: 0.7903 - val_loss: 0.4595
Epoch 13/100
18/18 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.8279 - loss: 0.3922 - val_accuracy: 0.8226 - val_loss: 0.4681

Training stopped at epoch: 13
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 13ms/step
```





10. Conclusion

This experiment demonstrates the end-to-end construction and tuning of an Artificial Neural Network using Keras and TensorFlow on the Pima Indians Diabetes dataset. The baseline ANN (Input → Dense(64) → Dense(32) → Sigmoid) achieves 70.13% accuracy, providing a competitive starting point for a dataset of only 768 samples with moderate class imbalance (500 vs 268). The network learns non-linear decision boundaries that simpler linear models cannot capture, evidenced by meaningful precision and recall scores for both classes despite the relatively small training set.

Keras Tuner's RandomSearch across 20 trials identifies a superior architecture: a wider first layer (128 units) with 0.2 dropout, followed by a standard second layer (32 units), trained at a higher learning rate (0.005) with early stopping. This configuration improves accuracy to 74.68% and, more importantly, raises diabetes precision from 0.57 to 0.65 — reducing false alarms by 32% (25 → 17 false positives). The early stopping at epoch 13 demonstrates that the optimal configuration converges rapidly, avoiding the over-training that afflicts the default 100-epoch baseline. Collectively, the results confirm that architectural width, targeted dropout regularisation, and learning rate selection are the three most impactful ANN hyperparameters for tabular binary classification tasks.